Phase 5 — TRANSFORM

Created	@March 1, 2026 1:17 PM
Tags	

ภาพรวมของ Transform Task

```
/tmp/fraud_etl/raw_{key}.parquet
    ↓  PySpark อ่าน Parquet
    ↓  Hash SHA-256 (cc_num, first, last, dob) + Rename Col
umns
    ↓  แปลง DateTime + Rename Columns
    ↓  Filter amt
    ↓  Select columns
/tmp/fraud_etl/clean_{key}.parquet
```

STEP 1 : เพิ่ม Transform function ใน DAG

หาบรรทัด `# ====================` ของ DAG Definition แล้วเพิ่ม Transform function **ก่อน** DAG Definition:

```
# ==========================================================
====================
# TASK 2 — TRANSFORM
# ==========================================================
====================
def transform(dataset_key: str, **kwargs):
    ti       = kwargs['ti']
    in_path  = ti.xcom_pull(key=f'parquet_raw_{dataset_ke
y}')
    out_path = f'{PARQUET_DIR}/clean_{dataset_key}.parquet'

    t_start = time.perf_counter()

    # สร้าง SparkSession
```

```
    spark = (SparkSession.builder
             .appName('fraud_etl')
             .config('spark.jars', JDBC_JAR)
             .getOrCreate())

    # สร้าง SHA-256 UDF
    hash_udf = udf(
        lambda x: hashlib.sha256(str(x).encode()).hexdigest
()
                  if x is not None else None,
        StringType()
    )

    df = spark.read.parquet(in_path)

    df = (df
        # Step 1: Hash ข้อมูลส่วนตัว + Rename Columns
        .withColumn('cc_num_hash', hash_udf(col('cc_num')))
        .withColumn('first_hash',  hash_udf(col('first')))
        .withColumn('last_hash',   hash_udf(col('last')))
        .withColumn('dob_hash',    hash_udf(col('dob')))

        # Step 2: แปลง DateTime + Rename Columns
        .withColumn('trans_datetime',
                    to_timestamp(col('trans_date_trans_tim
e'),
                                 'yyyy-MM-dd HH:mm:ss'))

        # Step 3: Filter amt ผิดปกติ
        .filter(col('amt').isNotNull())
        .filter(col('amt') > 0)

        # Step 4: Select เฉพาะ column ที่ต้องการ Load
        .select(
            col('trans_datetime'),
            col('cc_num_hash'),
            col('merchant'),
            col('category'),
```

```
            col('amt').cast('double'),
            col('first_hash'),
            col('last_hash'),
            col('gender'),
            col('city'),
            col('state'),
            col('city_pop').cast('int'),
            col('job'),
            col('dob_hash'),
            col('trans_num'),
            col('is_fraud').cast('int')
        )
    )

    df.write.mode('overwrite').parquet(out_path)
    spark.stop()

    transform_sec = time.perf_counter() - t_start

    ti.xcom_push(key=f'transform_sec_{dataset_key}', value=
transform_sec)
    ti.xcom_push(key=f'parquet_clean_{dataset_key}', value=
out_path)

    print(f'[Transform] {dataset_key}: {transform_sec:.3f}
s')
```

Save

STEP 2 : แก้ DAG Definition ให้รวม Transform

หา `DAG Definition` แล้วแก้ for loop:

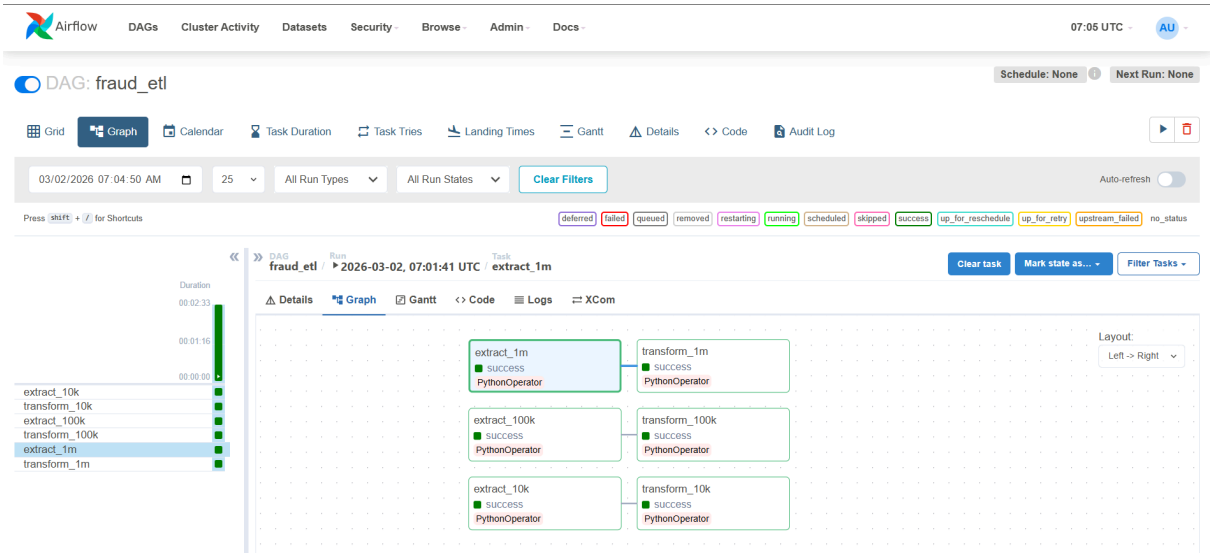
```
with DAG(
    dag_id='fraud_etl',
    default_args=default_args,
    schedule_interval=None,
    start_date=datetime(2024, 1, 1),
```

```
    catchup=False
) as dag:

    for key in ['10k', '100k', '1m']:
        extract_task = PythonOperator(
            task_id=f'extract_{key}',
            python_callable=extract,
            op_kwargs={'dataset_key': key}
        )
        transform_task = PythonOperator(
            task_id=f'transform_{key}',
            python_callable=transform,
            op_kwargs={'dataset_key': key}
        )

        # Chain: extract → transform
        extract_task >> transform_task
```

บันทึก: `Ctrl+O` → `Enter` → `Ctrl+X`

STEP 3 : Trigger DAG

เปิดเบราว์เซอร์ไปที่ `http://localhost:8080` แล้วคลิก Trigger DAG



-

STEP 4 : ตรวจสอบผลลัพธ์

```
source ~/airflow_env/bin/activate
watch -n 3 'ls -lh /tmp/fraud_etl/'
```

ผลที่ควรเห็น:

```
raw_10k.parquet      ← Extract output (มีอยู่แล้ว)
raw_100k.parquet
raw_1m.parquet
clean_10k.parquet    ← Transform output (ใหม่)
clean_100k.parquet
clean_1m.parquet
```

ตรวจ column ว่า Transform ถูกต้องไหม:

```
python3 - <<'EOF'
import pyarrow.parquet as pq
for key in ['10k', '100k', '1m']:
    table = pq.read_table(f'/tmp/fraud_etl/clean_{key}.parq
uet')
    print(f'\n{key}: {table.num_rows} rows')
    print(f'Columns: {table.column_names}')
EOF
```

ผลที่ควรได้:

```
10k: 10000 rows
Columns: ['trans_datetime', 'cc_num_hash', 'merchant', 'cat
egory', 'amt', 'first_hash', 'last_hash', 'gender', 'city',
'state', 'city_pop', 'job', 'dob_hash', 'trans_num', 'is_fr
aud']

100k: 100000 rows
Columns: ['trans_datetime', 'cc_num_hash', 'merchant', 'cat
egory', 'amt', 'first_hash', 'last_hash', 'gender', 'city',
'state', 'city_pop', 'job', 'dob_hash', 'trans_num', 'is_fr
aud']
```

```
1m: 1000000 rows
Columns: ['trans_datetime', 'cc_num_hash', 'merchant', 'cat
egory', 'amt', 'first_hash', 'last_hash', 'gender', 'city',
'state', 'city_pop', 'job', 'dob_hash', 'trans_num', 'is_fr
aud']
```

สรุปสิ่งที่ทำใน TRANSFORM

Input รับ path ของ `raw_{key}.parquet` มาจาก Extract ผ่าน XCom

Step 1 — Hash ข้อมูลส่วนตัว

เข้ารหัส SHA-256 ทั้ง 4 column แล้วสร้าง column ใหม่แทน

```
cc_num  → cc_num_hash
first   → first_hash
last    → last_hash
dob     → dob_hash
```

Step 2 — แปลง DateTime

```
trans_date_trans_time (String) → trans_datetime (DateTime)
```

Step 3 — Filter

ตัดแถวที่ข้อมูลผิดปกติออก

```
amt เป็น NULL  → ตัดออก
amt <= 0       → ตัดออก
```

Step 4 — Select

เลือกเฉพาะ 15 column ที่ตรงกับ Schema ของ MySQL โดยทิ้ง column เก่าอย่าง `cc_num`, `first`, `last`, `dob`, `trans_date_trans_time` ออก

Output บันทึกเป็น `clean_{key}.parquet` แล้วส่ง path และเวลาที่ใช้ผ่าน XCom ให้ Task Load ต่อไป